

Controller ve Routing

Minimal endpoint'ten düzenli controller yapısına geç. İstekleri action metotlarına yönlendir ve doğru HTTP cevabını seç.

Yaklaşık	Odak	Çıktı
150–210 dk	Attribute routing	ProductsController

Bugünün çalışma kuralı

Önce tek çalışan controller oluştur. Liste, route parametresi ve hata cevaplarını aynı anda yazmaya çalışma.

Bitiş ölçütü

- Controller'ın sorumluluğunu açıklamak
- AddControllers ve MapControllers farkını bilmek
- GET /api/products endpoint'ini controller ile yazmak
- Route parametresiyle ürün bulup 200 veya 404 döndürmek

1. Controller neden var?

Minimal API'de endpoint'leri doğrudan Program.cs içine yazdık. Proje büyüdükçe bu dosya kalabalıklaşır. Controller, ilgili HTTP işlemlerini ayrı bir class'ta toplar.

Minimal API	Controller
Küçük örnek ve servislerde hızlı	Büyüyen API'de düzenli yapı
HttpGet doğrudan Program.cs'de	HttpGet action metodu içinde
Az dosya	Konulara göre ayrı controller'lar

İstek akışı



Action

HTTP isteğini karşılayan public controller metodudur. Örneğin GetAll, bütün ürünleri döndüren bir action olabilir.

Önerilen klasör

```

Gun8-WebApi/
├── Controllers/
│   └── ProductsController.cs
├── Program.cs
└── Gun8-WebApi.csproj
  
```

2. Controller altyapısını açmak

Controller class'ı yazmak yetmez. ASP.NET Core'a controller servislerini kaydetmesini ve controller route'larını eşlemesini söylemeliyiz.

Program.cs

```
var builder = WebApplication.CreateBuilder(args);  
  
builder.Services.AddControllers();  
  
var app = builder.Build();  
  
app.MapControllers();  
  
app.Run();
```

Satır	Görevi
AddControllers	Controller servislerini uygulamaya kaydeder.
MapControllers	Attribute ile tanımlanan route'ları etkinleştirir.

İkisi de gerekli

AddControllers hazırlığı yapar; MapControllers gelen isteği uygun controller action'ına yönlendirir.

Eski minimal endpoint'ler

MapGet endpoint'lerini istersen şimdilik koruyabilirsin. Controller endpoint'leriyle aynı metot + adresi kullanmadıkları sürece birlikte çalışırlar.

3. Controller anatomisi

```
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    [HttpGet]
    public IActionResult GetAll()
    {
        return Ok(new[] { "Klavye", "Mouse" });
    }
}
```

Parça	Anlamı
[ApiController]	API controller davranışlarını etkinleştirir.
[Route(...)]	Controller'ın temel adresini belirler.
ControllerBase	Ok ve NotFound gibi yardımcıları sağlar.
[HttpGet]	Metodun GET isteğini karşıladığını belirtir.
IActionResult	Farklı HTTP cevapları döndürmeye izin verir.

[controller] nasıl çözülür?

Class adı ProductsController ise sonundaki "Controller" çıkarılır ve adres /api/products olur.

Test

```
curl.exe -i http://localhost:PORT/api/products
```

4. İlk küçük çalışan controller

Önce veri modeli olmadan tek bir sağlık endpoint'i oluştur. Böylece controller ve routing altyapısını ayrı olarak doğrularsın.

Controllers/HealthController.cs

```
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class HealthController : ControllerBase
{
    [HttpGet]
    public IActionResult Get()
    {
        return Ok(new
        {
            Status = "Healthy",
            Time = DateTime.Now
        });
    }
}
```

Oluşturulan endpoint

GET /api/health

Beklenen cevap

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "status": "Healthy",
  "time": "2026-09-08T18:00:00"
}
```

Ok(...)

200 OK durum kodu ile birlikte response body döndürür. ASP.NET Core nesneyi otomatik JSON'a çevirir.

5. ProductsController ve liste

Controller içinde şimdilik bellekte duran sabit bir ürün listesi kullanacağız. Veritabanı daha sonraki günlerde eklenecek.

Model örneği

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public decimal Price { get; set; }
}
```

Action iskeleti

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private static readonly List<Product> Products = new()
    {
        new Product { Id = 1, Name = "Klavye", Price = 850 },
        new Product { Id = 2, Name = "Mouse", Price = 450 }
    };

    [HttpGet]
    public IActionResult GetAll()
    {
        // Products listesini 200 OK ile döndür.
        return /* ... */;
    }
}
```

static readonly

Bu eğitimde listenin controller nesneleri arasında korunmasını sağlar. Gerçek uygulamada veri kaynağı olarak veritabanı ve service katmanı kullanılacaktır.

Görev

GetAll action'ındaki eksik dönüşü tamamla ve GET /api/products adresini test et.

6. Route parametresiyle ürün bulmak

Adresin değişen bölümünü süslü parantezle tanımlarız:

```
[HttpGet("{id}")]
```

Bu route şu istekleri karşılar:

```
GET /api/products/1
GET /api/products/2
GET /api/products/99
```

Parametre eşleşmesi

```
[HttpGet("{id}")]
public IActionResult GetById(int id)
{
    // URL'deki {id}, metodun id parametresine gelir.
}
```

İsimler eşleşmeli

Route içindeki {id} ile metod parametresi id aynı adı taşımalıdır.

Görev iskeleti

```
[HttpGet("{id}")]
public IActionResult GetById(int id)
{
    Product? product = Products.FirstOrDefault(
        item => /* id karşılaştırması */);

    if (/* ürün bulunamadı mı? */)
    {
        return NotFound(new
        {
            Message = "Ürün bulunamadı."
        });
    }

    return /* ürünü 200 OK ile döndür */;
}
```

Beklenen sonuç

- GET /api/products/1 200 OK ve Klavye
- GET /api/products/99 404 Not Found ve hata JSON'u

7. Test, hata ayıklama ve Git

Test komutları

```
curl.exe -i http://localhost:PORT/api/health
curl.exe -i http://localhost:PORT/api/products
curl.exe -i http://localhost:PORT/api/products/1
curl.exe -i http://localhost:PORT/api/products/99
```

Sık hatalar

Belirti	Kontrol
Bütün adresler 404	AddControllers ve MapControllers var mı?
Controller bulunmuyor	Class public ve adı Controller ile bitiyor mu?
Route farklı	[Route] ve [HttpGet] değerlerini kontrol et.
Kod değişmedi	Kaydet, Ctrl+C, dotnet run.

Mülakat notları

- Controller, ilgili HTTP action'larını bir class'ta toplar.
- Routing, isteği uygun controller/action'a yönlendirir.
- ControllerBase, Ok ve NotFound gibi yardımcı metotlar sunar.
- IActionResult farklı durum kodları ve body'ler döndürmeyi sağlar.

Git

```
cd "C:\Users\Administrator\.NetEgitim"
git add .
git commit -m "Gün 9 controller ve routing tamamlandı"
git push
```

Gün 9 tamamlanma ölçütü

Liste endpoint'i 200; var olan ürün 200; olmayan ürün 404 döndürüyorsa hedef tamamdır.